RozRides — Complete Booking Flow: Every Step, Every Scenario

HOW TO READ THIS DOCUMENT

Every flow is written from the perspective of what the USER SEES, what the APP DOES, and what FIRESTORE RECEIVES at each step. Every possible scenario including happy paths, edge cases, errors, and failures is covered.

When you send this to Claude in your IDE, it can use this as a checklist to verify every screen, every function, and every database write is working correctly.

SECTION 1 — THE RENTER JOURNEY: FINDING A CAR

Flow 1.1 — Renter Opens the App and Browses

What the user sees:
Home screen loads with a list of car cards. Each card shows the car photo, name, location, price per day, star rating, and either a green "Insured" badge or orange "Not Insured" badge.

What the app does:
Queries the cars collection where status == "approved" and isAvailable == true. Fetches in real-time.

What Firestore does:
Returns approved car documents ordered by createdAt descending.

Scenarios:

Scenario A — Cars load successfully:
Cards render. User scrolls and browses.

Scenario B — No cars available:
Empty state illustration and text "No cars available right now. Check back soon." appears.

Scenario C — Network error:
Error state shows with a retry button. Tapping retry re-runs the query.

Scenario D — User is not CNIC verified:
Cars still display normally. The verification gate only triggers when they try to book, not when browsing. No banner or warning on the home screen about this.

Flow 1.2 — Renter Taps a Car Card

What the user sees:
Navigates to the Car Detail Screen. Screen loads with:

- Photo carousel at the top
- Car name and location
- Insurance badge (green Insured or orange Not Insured)
- Star rating and review count
- Host info section with host photo, name, rating, and response time
- Car specs (transmission, fuel, seats, features)
- Pricing section showing price per day and security deposit amount
- Calendar widget
- "How Cash Works" explainer section
- Car rules
- Reviews section
- Disabled button at the bottom reading "Select Dates to Continue"

What the app does:
Fetches the full car document from cars/{carId}. Simultaneously fetches all confirmed and active bookings for this car to populate the calendar's blocked dates. Fetches host user document for host info display.

What Firestore does:
Three reads:

1. cars/{carId} — full car document
2. bookings collection query where carId == thisCarId AND status in ["confirmed", "active"]
3. users/{hostId} — host profile data

Scenarios:

Scenario A — Car loads successfully:
All sections render. Calendar shows blocked dates in gray.

Scenario B — Car has no bookings yet:
Calendar shows all dates available (no grayed dates).

Scenario C — Car is fully booked for the next 30 days:
Calendar shows the next 30 days all grayed out. User can scroll forward to find available dates.

Scenario D — Car document missing fields (older listing):
App handles nulls gracefully. Missing insurance field shows nothing. Missing security deposit shows "Contact host."

Scenario E — Host account was deleted:
Host info section shows "Host unavailable." Car is still displayable.

Flow 1.3 — Renter Selects Dates on the Calendar

What the user sees:
Calendar widget is interactive. User taps a start date — that date highlights in green. User taps an end date — the range between start and end highlights in light green. Dates that are blocked show in gray and are non-tappable.

Below the calendar, a pricing breakdown appears immediately:

- "X days × PKR Y = PKR Z"
- "Security deposit: PKR A"
- "AT PICKUP — bring: PKR A (deposit)"
- "AT RETURN — pay: PKR Z (rental)"
- "AT RETURN — receive: PKR A (deposit back)"

The bottom button activates and changes text to "REQUEST TO BOOK — PKR Z total"

What the app does:
On each date tap, runs the PricingCalculator.calculate() function locally with the selected dates. No network call needed — pricing data is already fetched with the car document. Updates the UI reactively.

Scenarios:

Scenario A — Renter selects valid available dates:
Pricing appears. Button activates. Happy path.

Scenario B — Renter taps a blocked date:
Nothing happens. That date is non-interactive. No error shown — the gray visual is sufficient communication.

Scenario C — Renter taps end date before start date:
App treats the second tap as the new start date and waits for an end date selection.

Scenario D — Renter selects a range that includes a blocked date in the middle:
The app detects this and shows a snackbar: "Your selected range includes unavailable dates. Please choose different dates." The selection resets.

Scenario E — Renter selects just one day (same start and end date):
App shows an error: "Minimum rental period is 1 day. Please select a return date after your pickup date." The button stays disabled.

Scenario F — Renter selects more than 30 days (if host has set a max):
If car document has maxRentalDays field set, show: "This host allows a maximum rental of X days." Selection resets to the max allowed.

SECTION 2 — SUBMITTING THE BOOKING REQUEST

Flow 2.1 — Renter Taps "REQUEST TO BOOK"

What the user sees:
Navigates to the Booking Summary Screen. This screen shows:

- Car thumbnail and name
- Host name
- Pickup date and return date formatted clearly
- Duration
- Their own profile photo and name with CNIC status badge
- Full cash payment schedule broken into two sections (AT PICKUP and AT RETURN)
- Optional message text field

- Cancellation policy note
- Cash disclaimer note
- "SEND BOOKING REQUEST" button at the bottom

What the app does:
Passes the car data, selected dates, and computed CashPricingBreakdown to the Booking Summary Screen. No network calls yet on this screen load.

What Firestore does:
Nothing yet. No writes have happened.

Flow 2.2 — CNIC Check on Booking Summary Screen Load

What the app does immediately when Booking Summary Screen opens:
Fetches users/{currentUserId} and checks cnicStatus field.

Scenarios:

Scenario A — CNIC is approved:
Screen renders normally. No warnings. Button is enabled.

Scenario B — CNIC is pending:
A red warning banner appears below the profile section: "Your identity verification is pending review. You cannot book until it is approved." The "SEND BOOKING REQUEST" button is disabled and grayed out. A "Go to Profile" link is shown.

Scenario C — CNIC is rejected:
Red banner: "Your identity verification was rejected. Please resubmit your documents." Button disabled. "Go to Profile" link shown.

Scenario D — CNIC has never been submitted:
Red banner: "You must complete identity verification before booking. This keeps everyone safe." Button disabled. "Complete Verification" link shown.

Flow 2.3 — Renter Fills Optional Message and Taps "SEND BOOKING REQUEST"

What the user sees:
Loading indicator appears on the button. Button text changes to "Sending..." Button becomes non-tappable to prevent double submission.

What the app does — step by step:

Step 1: Final CNIC check
Calls users/{currentUserId} one more time. If cnicStatus is not "approved", throws "CNIC_NOT_VERIFIED" before any booking is created.

Step 2: Final availability check
Calls BookingService.checkAvailability(carId, startDate, endDate). This queries all bookings for this car with status "confirmed" or "active" and runs the overlap formula on each:

text

overlap = newStart < existingEnd AND newEnd > existingStart

If any overlap found, throws "DATES_UNAVAILABLE".

Step 3: Create booking document
Generates a new Firestore document reference with auto-ID. Builds the full booking map with status "pending" and expiresAt = now + 24 hours. Writes to Firestore.

Step 4: Write first timeline entry
Writes to bookings/{bookingId}/timeline/ a new document: status "pending", triggeredBy = renterId, note = "Booking request submitted by renter", timestamp = server timestamp.

Step 5: Write notification for host
Writes to notifications/ collection a document: userId = hostId, type = "new_booking_request", title = "New Booking Request", body = "[Renter name] wants to rent your [car name] from [startDate] to [endDate]", bookingId = the new bookingId, isRead = false, createdAt = server timestamp.

Step 6: Navigate to success screen
Navigates to BookingConfirmedScreen replacing the current route (so back button cannot return to summary screen).

What Firestore receives:

text

bookings/{newBookingId}:

  bookingId: "abc123..."

  carId: "car456..."

  hostId: "host789..."

renterId: "renter012..."

carName: "Toyota Corolla 2020"

carPhoto: "https://..."

carLocation: "DHA Phase 5, Lahore"

startDate: Timestamp(Jan 15 2025)

endDate: Timestamp(Jan 17 2025)

totalDays: 2

pricePerDay: 3500

totalRent: 7000

securityDeposit: 10000

cashPayments: { all false/zero/null }

status: "pending"

messageToHost: "Hi, is morning pickup okay?"

cancellationPolicy: "flexible"

reviewStatus: { renterSubmitted: false, hostSubmitted: false }

tripStartedAt: null

tripEndedAt: null

createdAt: ServerTimestamp

updatedAt: ServerTimestamp

expiresAt: Timestamp(Jan 14 2025 + 24hrs from creation)

bookings/{newBookingId}/timeline/{autoId}:

status: "pending"

triggeredBy: "renter012..."

timestamp: ServerTimestamp

note: "Booking request submitted by renter"

notifications/{autoId}:

userId: "host789..."

type: "new_booking_request"

bookingId: "abc123..."

title: "New Booking Request"

body: "Ali Hassan wants to rent your Toyota Corolla..."

isRead: false

createdAt: ServerTimestamp

Error Scenarios:

Scenario A — CNIC not approved (caught at Step 1):
Loading stops. Dialog appears: "Identity Verification Required — You must complete and be approved for identity verification before booking a car. This keeps everyone on the platform safe." Two buttons: "Go to Profile" and "Cancel". No booking document is created.

Scenario B — Dates became unavailable between browsing and submitting (caught at Step 2):
Loading stops. Dialog appears: "These dates were just booked by someone else. Please go back and select different dates." One button: "Select New Dates" which pops back to the car detail screen. No booking document is created.

Scenario C — Network failure during write (Step 3 fails):
Loading stops. Snackbar appears: "Something went wrong. Please check your connection and try again." Button re-enables. No partial booking is created.

Scenario D — User taps button twice very fast:
Button is disabled immediately on first tap. Second tap does nothing. Only one booking is created.

Flow 2.4 — Booking Confirmed Screen

What the user sees:
Large green checkmark at top. "Request Sent!" title. Explanation text about 24-hour window. A cash reminder card: "Have PKR 10,000 ready — this is your security deposit if

the host accepts." Expiry time shown. Two buttons: "VIEW MY BOOKINGS" and "BACK TO HOME".

Scenarios:

Scenario A — User taps "VIEW MY BOOKINGS":
Navigates to MyBookingsScreen where the new pending booking appears at the top.

Scenario B — User taps "BACK TO HOME":
Pops all routes back to home screen.

Scenario C — User presses phone's back button:
Since the route was replaced (not pushed), back button goes to home screen. The summary screen is not in the navigation stack.

SECTION 3 — THE HOST RECEIVES THE REQUEST

Flow 3.1 — Host Receives Push Notification

What the host sees:
Push notification appears on their device even if the app is closed:
Title: "🚗 New Booking Request!"
Body: "Ali Hassan wants to rent your Toyota Corolla — Jan 15 to Jan 17"

What the app does when notification is tapped:
If app is closed: opens app and navigates directly to RequestDetailScreen for that bookingId.
If app is open: navigates directly to RequestDetailScreen.
If app is in background: brings app to foreground and navigates.

What Firestore does:
The notification document already exists (written in Flow 2.3 Step 5). The notification handler reads the bookingId from the notification data payload to know where to navigate.

Flow 3.2 — Host Opens Incoming Requests Screen

What the host sees:
A screen with tabs "Pending (X)" and "All". Under Pending tab, a list of booking request cards. Each card shows:

- Renter photo and name
- CNIC verification badge (green if approved, red if not)
- Car name and small photo
- Dates and duration
- "Your rental income: PKR 7,000" (the totalRent)
- Expiry countdown: "Expires in 18 hours" — red if under 4 hours
- Two buttons: DECLINE (outlined red) and ACCEPT (filled green)

What the app does:
Real-time stream listener on bookings where hostId == currentUserId AND status == "pending". Updates automatically as bookings expire or get cancelled by renters.

Scenarios:

Scenario A — No pending requests:
Empty state: "No pending requests. When renters request your cars, they'll appear here."

Scenario B — Multiple pending requests for same car on overlapping dates:
Both cards show. Host can only accept one. When one is accepted, the other auto-declines (handled in accept flow).

Scenario C — A request shows renter is not CNIC verified:
Red "Unverified" badge on the renter's card. Host can still accept or decline — the CNIC check is the renter's responsibility. The badge is informational for the host.

Scenario D — Request expires while host is looking at the screen:
The countdown reaches zero. The card automatically disappears from the list because the real-time stream no longer includes expired bookings. No action needed from host.

Flow 3.3 — Host Taps a Request Card to View Detail

What the host sees:
Full RequestDetailScreen with:

- Renter's large profile photo, name, CNIC badge
- Member since date
- Number of previous completed trips as renter

- Renter's average renter rating (stars) if they have past reviews
- Car name and photo
- Exact pickup and return dates and times
- Duration
- Cash income breakdown showing what host collects and what they return
- Renter's message if they wrote one, or "No message from renter"
- Two full-width buttons at bottom: DECLINE REQUEST and ACCEPT REQUEST

What the app does:
Fetches renter's user document. Queries completed bookings where renterId == thisRenterId to get trip count. Queries reviews where revieweeId == thisRenterId and type == "host_to_renter" to calculate rating.

Scenarios:

Scenario A — Renter is new with no previous trips:
Shows "0 previous trips" and no rating display (not "0 stars" — just omit the rating section).

Scenario B — Renter has previous trips and reviews:
Shows count and average rating.

Scenario C — Booking expired while host was reading the detail:
The real-time listener updates the screen's booking data. A banner appears: "This request has expired and is no longer active." Both buttons become disabled.

Scenario D — Renter cancelled the request while host was reading:
Same — banner: "The renter cancelled this request." Both buttons disabled.

Flow 3.4 — Host Taps "ACCEPT REQUEST"

What the host sees:
A bottom sheet slides up from the bottom with:

- "Accept this booking?" title
- Summary: renter name, car name, dates
- Cash reminder: "Remember to collect PKR 10,000 deposit at pickup, and PKR 7,000 rent at return"

- Two buttons: "Cancel" and "Yes, Accept"

When host taps "Yes, Accept":

Loading indicator appears on the button.

What the app does — step by step in a Firestore Transaction:

Step 1: Read the booking document inside the transaction
Verify status == "pending". If not pending anymore, abort.

Step 2: Read the car document inside the transaction
Check bookedDateRanges array for any overlap with this booking's dates using the interval overlap formula. This is the race condition protection — if two hosts somehow share a listing (edge case) or if the system has a bug, this catches it.

Step 3: Write to booking document
Update status to "confirmed", updatedAt to server timestamp.

Step 4: Write to car document
Append to bookedDateRanges array: { start: startDate, end: endDate, bookingId: bookingId }

Step 5: Write timeline entry
bookings/{bookingId}/timeline/ — status "confirmed", triggeredBy = hostId, note = "Host accepted the booking request"

After transaction succeeds (outside transaction):

Step 6: Auto-decline overlapping pending requests
Query bookings where carId == thisCarId AND status == "pending" AND bookingId != thisBookingId. For each result, run overlap check. For overlapping ones:

- Update status to "rejected"
- Set rejectionReason to "HOST_CHOSE_ANOTHER_BOOKING"
- Write notification to that renter: "Your request for [car name] was not accepted. The host chose a different booking for those dates. Browse similar cars."

Step 7: Create conversation document
Write to conversations/{bookingId} (use bookingId as conversationId):

text

conversationId: bookingId

bookingId: bookingId

carId: carId

participants: [hostId, renterId]

hostId: hostId

renterId: renterId

lastMessage: "Booking confirmed..."

lastMessageAt: ServerTimestamp

hostUnreadCount: 0

renterUnreadCount: 1

isActive: true

createdAt: ServerTimestamp

Step 8: Write system message to conversation
conversations/{bookingId}/messages/{autoId}:

text

senderId: "system"

text: "Booking confirmed for Toyota Corolla — Jan 15 to Jan 17.

You can now coordinate pickup details here."

type: "system"

createdAt: ServerTimestamp

isRead: false

Step 9: Write notification to renter
notifications/{autoId}: type "booking_confirmed", title "Booking Confirmed!", body "Ahmed accepted your booking for Toyota Corolla. Bring PKR 10,000 cash on Jan 15."

What Firestore receives after full accept flow:

text

bookings/{bookingId}:

status: "confirmed" (changed from "pending")

updatedAt: ServerTimestamp (updated)

bookings/{bookingId}/timeline/{autoId}:

status: "confirmed"

triggeredBy: "host789..."

note: "Host accepted the booking request"

timestamp: ServerTimestamp

cars/{carId}:

bookedDateRanges: [...existing, { start: Jan15, end: Jan17, bookingId: "abc123" }]

conversations/{bookingId}:

(full document as above)

conversations/{bookingId}/messages/{autoId}:

(system message as above)

notifications/{autoId}:

userId: "renter012..."

type: "booking_confirmed"

...

Error Scenarios:

Scenario A — Booking was cancelled by renter while host was deciding:
Transaction Step 1 reads status as "cancelled". Transaction aborts. Bottom sheet closes. Snackbar: "This request is no longer available. The renter cancelled it." Host is returned to the requests list.

Scenario B — Network failure during transaction:
Transaction rolls back automatically (Firestore guarantee). Nothing is partially written. Snackbar: "Something went wrong. Please try again." Bottom sheet stays open.

Scenario C — Transaction succeeds but Step 6 (auto-decline) fails:
The booking is confirmed but some overlapping requests remain "pending". The Cloud Function's expiry job will eventually expire them. This is an acceptable degraded state — the overlap check on the next accept attempt will still catch conflicts.

Flow 3.5 — Host Taps "DECLINE REQUEST"

What the host sees:
A bottom sheet with a reason selector:

- "Car unavailable on these dates"
- "Car needs maintenance"
- "Renter profile incomplete"
- "I found a better booking"
- "Other" (reveals text field)

"Confirm Decline" button at bottom, disabled until a reason is selected.

When host selects reason and taps "Confirm Decline":

What the app does:

1. Updates bookings/{bookingId}:
    - status → "rejected"
    - rejectionReason → selected reason text
    - updatedAt → server timestamp
2. Writes timeline entry: status "rejected", triggeredBy = hostId, note = reason text
3. Writes notification to renter: title "Booking Request Declined", body "Ahmed declined your request for Toyota Corolla. Reason: [reason]. Browse other available cars."
4. Closes bottom sheet. Snackbar: "Request declined." Card disappears from pending list.

Scenarios:

Scenario A — Booking already expired before host could decline:
Update will succeed but status will be "expired" when the function tries to update "pending" to "rejected". Add a check: only update if current status is "pending". If already expired, just close the sheet and show "This request had already expired."

Scenario B — Host selects "Other" but leaves text field empty:
"Confirm Decline" button remains disabled. Text field shows a validation error: "Please describe your reason."

Flow 3.6 — 24-Hour Auto-Expiry System

This runs as a Cloud Function on a schedule every 30 minutes.

What the Cloud Function does:

Query 1: Find stale bookings
Queries bookings where status == "pending" AND expiresAt <= now. For each result:

- Updates status to "expired"
- Updates updatedAt
- Writes timeline entry: status "expired", triggeredBy "system", note "Host did not respond within 24 hours. Request automatically expired."
- Writes notification to renter: "Your request for [car name] expired — the host didn't respond in time. Browse similar available cars."

Query 2: Find bookings approaching expiry (for 20-hour warning)
Queries bookings where status == "pending" AND expiresAt <= now + 4 hours AND warningNotificationSent != true. For each result:

- Writes notification to host: "Booking request expiring soon — You have less than 4 hours to respond to [renter name]'s request for [car name]. After that it auto-expires."
- Updates warningNotificationSent: true on the booking document so this only sends once

Scenarios:

Scenario A — Host accepts right as the function runs:
The function queries pending bookings. The booking was just accepted so its status is now "confirmed". The query won't return it. No conflict.

Scenario B — Function runs but Firestore write fails for one booking:
That specific booking's expiry is retried on the next 30-minute run. Eventually succeeds.

Scenario C — Multiple function instances run simultaneously (Cloud Function cold start):
Use a Firestore transaction inside the expiry update to check status is still "pending" before updating. Prevents double-expiry.

SECTION 4 — RENTER MANAGES THEIR BOOKINGS

Flow 4.1 — Renter Opens My Bookings Screen

What the user sees:
Three tabs: Upcoming, Active, Past.

Upcoming tab shows pending and confirmed bookings.
Active tab shows bookings with status "active".
Past tab shows completed, cancelled, rejected, expired.

Each card has the appropriate status badge color and label from booking_status_utils.dart.

Scenarios:

Scenario A — Renter has no bookings:
Empty state on each tab: "Nothing here yet." on Upcoming tab has a "Browse Cars" button.

Scenario B — A pending booking's countdown shows 0 hours:
The card still shows until the Cloud Function marks it expired. Then it moves to Past tab automatically via real-time stream.

Scenario C — Renter has 10+ bookings:
List is scrollable. No pagination needed for MVP unless performance degrades.

Flow 4.2 — Renter Taps a Booking Card

What the user sees:
Booking Detail Screen. This screen shows different content based on the booking's current status.

PENDING STATUS view:

- Orange status banner: "⏱️ Awaiting Host Response — Expires in X hours"
- Car details
- Trip dates
- Host info with message button (grayed out — cannot message until confirmed)
- Full cash schedule (all items shown as pending/gray circles)
- "Cancel Request" button (outlined red)
- Booking timeline showing one entry: "Request Submitted"

CONFIRMED STATUS view:

- Green status banner: "✅ Confirmed — Your trip starts in X days"
- Car details
- Trip dates
- Host info with active "Message Host" button
- Cash schedule with deposit item still gray (not paid yet), other items pending
- "Cancel Booking" button (outlined red) with cancellation policy note below it
- Booking timeline showing: "Request Submitted → Confirmed"

ACTIVE STATUS view:

- Blue status banner: "🚙 Trip Active — Return in X days"
- Car details
- "View Active Trip" full-width button (navigates to ActiveTripScreen)
- Cash schedule: deposit item shows green checkmark (paid), rent item still gray (not paid yet)
- Booking timeline showing: "Request Submitted → Confirmed → Trip Started"

COMPLETED STATUS view:

- Gray status banner: "✓ Trip Completed"
- Car details
- Full cash schedule all green checkmarks
- "Leave a Review" button if reviewStatus.renterSubmitted == false
- "View Trip Summary" button showing full inspection comparison
- Complete booking timeline

REJECTED STATUS view:

- Red status banner: "✗ Declined by Host"
- Rejection reason shown: "Reason: Car needs maintenance"
- "Browse Other Cars" button
- Simple timeline

EXPIRED STATUS view:

- Red status banner: "✗ Request Expired — Host did not respond"
- "Browse Other Cars" button
- Simple timeline

Flow 4.3 — Renter Cancels a Pending Request

What the user sees:
From booking detail or booking card, taps "Cancel Request". Navigates to CancellationScreen.

Screen shows:

- Car name and dates reminder
- Information box: "Since no payment has been made, there's nothing to refund. The booking will simply be cancelled."
- Reason selector (required)
- "Confirm Cancellation" button

When confirmed:

What the app does:

1. Updates bookings/{bookingId}:
    - status → "cancelled"
    - cancellationReason → selected reason
    - updatedAt → server timestamp
2. Note: No need to remove from cars/{carId}/bookedDateRanges because pending bookings were never added there. Only confirmed bookings block dates.
3. Writes timeline entry: status "cancelled", triggeredBy = renterId, note = reason
4. Writes notification to host: "Ali Hassan cancelled their booking request for Toyota Corolla (Jan 15–17)."
5. Navigates back to My Bookings screen with snackbar: "Request cancelled."

Scenarios:

Scenario A — Booking was accepted by host right as renter cancels:
A Firestore transaction should be used. Read the status inside transaction — if it's now "confirmed" instead of "pending", abort and show: "This booking was just accepted by the host. You can still cancel but should message the host to coordinate." Show the confirmed booking detail instead.

Flow 4.4 — Renter Cancels a Confirmed Booking

What the user sees:
Navigates to CancellationScreen. Screen shows the same layout but with different information text:

"This booking has been confirmed. Cancelling will free up the dates for other renters."

If within 24 hours of pickup:
"⚠ Late Cancellation Warning: You are cancelling within 24 hours of your pickup. This will be noted on your renter profile."

When confirmed:

What the app does:

1. Updates booking status to "cancelled"

2. **Removes the date range from** cars/{carId}/bookedDateRanges — this is critical so the car becomes bookable again
3. If within 24 hours of pickup: increments renterStats.lateCancellations counter on the renter's user document
4. Writes timeline entry
5. Writes notification to host: "Ali Hassan cancelled their confirmed booking for Toyota Corolla (Jan 15–17). Those dates are now available again."

Scenarios:

Scenario A — Host had already arranged plans around the booking (informational only):
App cannot know this. The notification to the host is the only communication mechanism.

Scenario B — Removal from bookedDateRanges fails:
This is critical — if this fails, the car remains blocked even though booking is cancelled. Wrap in a transaction. If removal fails, retry. Log an error to admin alerts.

SECTION 5 — THE HOST MANAGES CONFIRMED BOOKINGS

Flow 5.1 — Host Opens Their Bookings Screen

What the host sees:
Four tabs: Requests (pending count badge), Upcoming (confirmed), Active, Past.

Each tab shows booking cards from host's perspective showing renter info and cash they'll collect.

For the Upcoming tab:
Confirmed bookings. Each card shows:

- Renter photo and name
- Car name
- Dates
- "Collect at pickup: PKR [deposit]"
- "Collect at return: PKR [rent]"

- "START HANDOVER" button — grayed and disabled if pickup date hasn't arrived yet, active green button on the day of pickup

---

Flow 5.2 — Pickup Day Arrives

What happens automatically:

On the morning of the pickup date (7:00 AM local time via Cloud Function):

Renter receives notification: " 🚗 Your trip starts today! Toyota Corolla pickup at [time]. Bring PKR 10,000 cash for the security deposit. Meeting at DHA Phase 5, Lahore."

Host receives notification: " 🚗 Ali Hassan arrives today! Collect PKR 10,000 security deposit at handover. Complete the inspection checklist together before handing over keys."

What changes on the host's booking card:
The "START HANDOVER" button becomes active (green and tappable).

SECTION 6 — THE HANDOVER: PRE-TRIP INSPECTION

Flow 6.1 — Host Taps "START HANDOVER"

Entry gate check:
If current time is more than 2 hours before the scheduled pickup time, show a warning dialog:
"It's a bit early. The scheduled pickup is at [time]. Are you sure you want to start the handover early?"
Two options: "Wait Until Pickup Time" and "Continue Anyway".

If renter is not CNIC verified (edge case check): show a blocking dialog: "The renter's identity verification is incomplete. Contact RozRides support before proceeding."

What the user sees:
Pre-Trip Inspection Screen loads. Step progress indicator at top shows "Step 1 of 7".

Flow 6.2 — Step 1: Security Deposit Confirmation

What the host sees:
Title: "Step 1 of 7 — Security Deposit"

Large text: "Collect cash from [renter name] before proceeding"
Expected amount displayed prominently: "PKR 10,000"
Number input field: "Amount received (PKR)" pre-filled with 10000
Checkbox: "I have received the cash deposit from [renter name]"
NEXT button disabled until checkbox is checked.

Scenarios:

Scenario A — Host collects exact amount:
Leaves pre-filled value. Checks box. Taps NEXT.

Scenario B — Host and renter agree on a different deposit amount:
Host edits the amount field. Both verbally agree. Host checks the box. The actual amount is what gets stored, not the expected amount from the listing. This handles edge cases like "I only had PKR 9,000 on me."

Scenario C — Renter refuses to pay the deposit:
Host should NOT check the box and should not proceed. The app cannot force this — it's a real-world interaction. The host should either wait or cancel the booking. If host wants to cancel at this point, they exit the inspection screen and cancel from booking detail. This counts as a host cancellation since the booking was confirmed.

Scenario D — Host accidentally taps NEXT without checking box:
NEXT button is disabled. Cannot proceed.

Flow 6.3 — Steps 2 Through 6: Area Photos

Each step (front, rear, left side, right side, interior) follows identical logic:

What the host sees:
Area name as title. Instruction to take a photo. Large camera button.

After photo taken: thumbnail shows with "Retake" option.

Below photo:
Toggle: "No damage" / "Damage exists"
If "Damage exists" selected: text field appears for description. Required if damage toggled on.

NEXT button disabled until photo is taken.

Scenarios:

Scenario A — Host takes a clear photo with no damage to note:
Selects "No damage". Taps NEXT. Happy path.

Scenario B — Pre-existing damage exists:
Host takes photo capturing the damage clearly. Selects "Damage exists". Types description: "Scratch on front bumper, approx 10cm, pre-existing." Both parties can see the screen and agree this is documented.

Scenario C — Camera permission not granted:
App requests camera permission. If denied, falls back to gallery picker. If neither is available, shows error: "Camera access is required for inspection. Please enable it in settings."

Scenario D — Photo upload fails:
On tapping NEXT, the photo is stored temporarily in local state. It is uploaded to Firebase Storage only at the final submission step, not on each step individually. This prevents step-by-step network dependency.

Scenario E — App crashes mid-inspection:
Current version does not persist inspection draft. Host must restart inspection. Future improvement: save draft to local storage. For MVP this is acceptable as the whole inspection takes about 5 minutes.

Flow 6.4 — Step 7: Fuel and Odometer

What the host sees:
Fuel level selector: Full | 3/4 | 1/2 | 1/4 | Empty
Current odometer reading: number input field.

Both required. NEXT button disabled until both filled.

Scenarios:

Scenario A — Both fields filled:
Taps NEXT. Proceeds to sign-off.

Scenario B — Odometer reading entered as letters:
Number input field prevents non-numeric input.

Scenario C — Odometer reading entered as 0:
Allow it — some hosts might not track odometer or it could genuinely read close to 0. No validation error.

Flow 6.5 — Sign-Off Screen

What the host sees:
"Handover Summary" heading.
Summary of all entered data:

- Deposit collected: PKR [amount] ✓
- Photos: 5 taken ✓
- Damage noted: "Scratch on front bumper" or "None"
- Fuel: Full
- Odometer: 45,231 km

"Host confirms the above is accurate:" — tappable area. When host taps, shows their name with a checkmark.

"Renter confirms the above is accurate:" — tappable area for renter.

"COMPLETE HANDOVER — START TRIP" button disabled until both have signed.

How signing works in practice:
Both people are physically present. The host taps their section, then hands the phone to the renter who taps their section. Simple tap-to-sign. No actual signature drawing required for MVP.

Scenarios:

Scenario A — Both sign:
Button activates. Host taps it.

Scenario B — Renter refuses to sign:
Host cannot proceed. This is a real-world dispute before the trip even starts. Host should not hand over keys. App cannot resolve this — it's a human issue.

Scenario C — Host signs but wants to edit something:
"Edit" button or back navigation. Goes back to the relevant step. Re-signing required after edits.

Flow 6.6 — "COMPLETE HANDOVER — START TRIP" is Tapped

What the host sees:
Loading indicator. Button text: "Starting trip..."

What the app does:

Step 1: Upload all 5 inspection photos to Firebase Storage
Path: inspections/{bookingId}/pre_trip/{area}_{timestamp}.jpg
Wait for all uploads to complete. Get download URLs back.

Step 2: Run Firestore transaction:

- Update bookings/{bookingId}:
    - status → "active"
    - tripStartedAt → server timestamp
    - cashPayments.depositPaidToHost → true
    - cashPayments.depositPaidAmount → amount entered in Step 1
    - cashPayments.depositPaidAt → server timestamp
    - updatedAt → server timestamp
- Write to bookings/{bookingId}/inspections/pre_trip:
  Full inspection document with all photo URLs, fuel, odometer, damage notes, deposit amount, signed = true for both
- Write timeline entry: status "active", note "Car handed over. Security deposit of PKR [amount] collected. Pre-trip inspection completed and signed by both parties."

Step 3: Post system message to conversation
conversations/{bookingId}/messages/: type "system", text "🚙 Trip started on Jan 15 at 10:23 AM. Security deposit of PKR 10,000 confirmed received. Safe travels!"
Update conversations/{bookingId}: lastMessage = that text, lastMessageAt = now, renterUnreadCount += 1

Step 4: Send notification to renter (if renter didn't initiate)
"Your trip for Toyota Corolla is now active! Return by Jan 17 at 10:00 AM."

Step 5: Navigate to Active Trip Screen

What Firestore receives:

text

bookings/{bookingId}:

status: "active"

tripStartedAt: ServerTimestamp

cashPayments.depositPaidToHost: true

cashPayments.depositPaidAmount: 10000

cashPayments.depositPaidAt: ServerTimestamp

updatedAt: ServerTimestamp

bookings/{bookingId}/inspections/pre_trip:

depositCollected: 10000

fuelLevel: "Full"

odometerReading: 45231

hostSigned: true

renterSigned: true

completedAt: ServerTimestamp

items: {

front: { hasDamage: true, notes: "Scratch on bumper", photoUrls: ["https://..."] }

rear: { hasDamage: false, notes: "", photoUrls: ["https://..."] }

leftSide: { hasDamage: false, notes: "", photoUrls: ["https://..."] }

rightSide: { hasDamage: false, notes: "", photoUrls: ["https://..."] }

interior: { hasDamage: false, notes: "", photoUrls: ["https://..."] }

}

bookings/{bookingId}/timeline/{autoId}:

status: "active"

triggeredBy: "host789..."

note: "Car handed over. Security deposit of PKR 10000 collected..."

timestamp: ServerTimestamp

Error Scenarios:

Scenario A — Photo upload fails:
Show error: "Failed to upload inspection photos. Please check your connection." Do not proceed with the transaction. Retry option shown.

Scenario B — Transaction fails after photos uploaded:
Photos are in Storage but booking status wasn't updated. The inspection data is not written. Host sees error and retries. On retry, photos are uploaded again (new paths with new timestamps). This creates orphaned photos in Storage — acceptable for MVP, clean up manually later.

Scenario C — Network cuts out during upload of 4th photo:
Upload of that photo fails. Error shown. Host retries. All 5 photos must succeed before proceeding.

SECTION 7 — THE ACTIVE TRIP

Flow 7.1 — Active Trip Screen (Renter View)

What the renter sees:
Car photo full width at top. Car name and registration.

Live countdown card:
"RETURN IN"
"1 day 14 hours 32 minutes"
Updates every minute via a Timer.

Return information:
"Return by: Friday, Jan 17 at 10:00 AM"
"Return location: DHA Phase 5, Lahore"

Cash reminder card:
"When you return the car:"
"💵 Pay PKR 7,000 rental to Ahmed"
"💵 Receive PKR 10,000 deposit back (if no damage)"

Quick action buttons:
"💬 Message Host" — opens ChatScreen for this booking's conversation
"📞 Emergency" — shows dialog with RozRides support contact
"📋 View Inspection Report" — read-only view of pre-trip inspection photos

Trip info section:
"Trip started: Jan 15 at 10:23 AM"
"Fuel at pickup: Full"
"Odometer at pickup: 45,231 km"

Car rules section from listing data.

Scenarios:

Scenario A — Renter presses back button:
WillPopScope triggers a dialog: "Are you sure you want to leave this screen? Your trip is still active." Options: "Stay" and "Go Back".

Scenario B — Return date passes and car not returned:
Countdown changes to: "⚠ OVERDUE — Please return the car immediately" in red.
The Cloud Function sends escalating notifications.

Flow 7.2 — Late Return Escalation

Triggered by Cloud Function checking every 30 minutes:

At exact return time (endDate):

- Renter notification: "Your trip has ended. Please return Toyota Corolla to Ahmed now."
- Host notification: "Ali Hassan's trip has ended. The car should be returned to you."

1 hour after endDate (if status still "active"):

- Renter: "Your car return is 1 hour overdue. Please contact Ahmed immediately."
- Host: "Car is 1 hour overdue. Contact Ali Hassan."
- Write lateNotifications.oneHourSent: true on booking to prevent duplicate

3 hours after endDate (if status still "active"):

- Both get final escalation notification

- Write to adminAlerts collection: "Overdue return alert: Booking [id] is 3 hours overdue."
- Write lateNotifications.threeHourSent: true on booking

Scenarios:

Scenario A — Car is returned and trip completed before 1-hour notification fires:
Cloud Function runs, checks status is now "completed". Skips notification. No action.

Scenario B — Renter and host verbally agree to extend the trip:
Current system has no extension mechanism for MVP. They would need to complete the current trip and create a new booking. Document this as a known limitation.

SECTION 8 — THE RETURN: POST-TRIP INSPECTION

Flow 8.1 — Host or Renter Initiates Return

Who can start it:
Either party can tap "Complete Return" from the booking detail screen. In practice, both should be physically present.

What the user sees:
Post-Trip Inspection Screen loads with step progress indicator.

Flow 8.2 — Step 1: Confirm Car Returned

"Confirm that [renter name] has returned the car."

Date and time: auto-filled as now. Both parties can see and agree on this.

Checkbox: "The car has been physically returned."

Scenarios:

Scenario A — Car returned on time:
Date shows as same as or before endDate. Green indicator: "Returned on time ✓"

Scenario B — Car returned late:
Date shows as after endDate. Yellow indicator: "Returned [X hours] late. This has been noted."

Flow 8.3 — Steps 2 Through 6: Area Comparisons

Each area step shows:

Top section: "AT PICKUP" — the pre-trip photo loaded from Firebase Storage. Read-only. Below it: the pre-trip damage note.

Bottom section: "NOW" — camera button to take new photo.

After new photo taken: side-by-side or stacked comparison of both photos.

Below comparison:
"Was there damage at pickup?" — shows pre-trip answer (read-only, cannot change)
"Is there NEW damage?" — toggle: "No new damage" / "Yes, new damage"
If new damage: required text field.

Scenarios:

Scenario A — No new damage on any area:
All areas show "No new damage" selected. Happy path to cash settlement.

Scenario B — New damage found on one area:
Toggle set to "Yes, new damage" with description. This triggers the damage path in cash settlement.

Scenario C — Pre-trip photo fails to load (Storage error):
Shows placeholder with: "Pre-trip photo unavailable. Reference the original inspection." The current photo can still be taken. Note this as a limitation.

Scenario D — Parties disagree on whether damage is new:
The pre-trip photo is the evidence. If pre-trip shows no damage in an area and post-trip shows damage, the comparison makes it visible. If both agree, proceed. If they disagree, proceed to dispute.

Flow 8.4 — Step 7: Fuel and Odometer Comparison

What the user sees:

"AT PICKUP: Fuel was Full"
"CURRENT FUEL LEVEL:" — selector

If current fuel is lower than pickup fuel:
Yellow warning banner: "⚠ Fuel level has decreased since pickup. This may result in a deduction from the deposit."

"AT PICKUP: Odometer was 45,231 km"
"CURRENT ODOMETER:" — number input

Auto-display: "Distance driven: [calculated] km"

Scenarios:

Scenario A — Fuel returned full:
No warning. Happy path.

Scenario B — Fuel not returned full:
Warning shown. This is noted in the comparison result and shown in cash settlement.

Scenario C — Odometer reading is lower than pre-trip (impossible physically):
Show validation error: "Odometer reading cannot be lower than at pickup (45,231 km)."

Flow 8.5 — Automatic Comparison Result

After step 7, the app runs compareInspections() locally:

text

```
For each area:

  if preTrip.hasDamage == false AND postTrip.hasDamage == true:

    → new damage found, add to issues list


If postTrip.fuelLevel worse than preTrip.fuelLevel:

  → fuel issue, add to issues list


kmDriven = postOdometer - preOdometer
```

Two possible outcomes:

Outcome A — No issues:
Shows: "✅ No new damage found! Great trip."

Sign-off section with host tap and renter tap.
Button: "PROCEED TO CASH SETTLEMENT"

Outcome B — Issues found:
Shows: "⚠ The following was noted during return inspection:"

- "New damage: [description] on [area]"
- "Fuel: returned at [level], was Full at pickup"
  Sign-off section.
  Button: "PROCEED TO CASH SETTLEMENT"

Both parties tap to sign. Both must sign before proceeding.

SECTION 9 — CASH SETTLEMENT

Flow 9.1 — Cash Settlement Screen: No Damage Path

What both parties see:

Title: "✅ All Clear — Final Cash Settlement"

Two step cards displayed sequentially:

Step 1 Card:
"Renter pays rental to host"
"PKR 7,000 cash"
"[Renter name] pays [Host name]"
Large checkbox: "Host confirms rental cash received" — host taps this

Step 2 Card (only visible after Step 1 checked):
"Host returns security deposit"
"PKR 10,000 cash — full refund, no damage"
"[Host name] returns to [Renter name]"
Large checkbox: "Renter confirms deposit received" — renter taps this

CLOSE TRIP button:
Disabled until both checkboxes checked. Once both checked, button activates.

Scenarios:

Scenario A — Both check their boxes and close:
Happy path. Proceeds to completeTrip().

Scenario B — Renter doesn't have exact change for rent:
App cannot handle this. Real-world problem. The checkbox is controlled by the host — host confirms when they receive an agreed amount. Host can tap checkbox at any agreed amount.

Flow 9.2 — Cash Settlement Screen: Damage Found Path

What both parties see:

Title: "⚠ Damage Found — Settlement"

Damage summary card showing all noted damage areas and fuel issue if any.

"Agree on a deduction:"
"Security deposit held: PKR 10,000"

Host enters deduction amount: "Damage deduction: PKR [input]"
Auto-calculates: "Renter receives back: PKR [10000 - deduction]"

Two buttons:
"RENTER AGREES — PROCEED" — renter taps this when they agree on the deduction amount
"RAISE A DISPUTE" — renter taps this if they don't agree

If renter agrees:
Proceeds to the two-step cash confirmation:
Step 1: Renter pays rent (same as no-damage path)
Step 2: Host returns partial deposit (PKR 10,000 - deduction amount)

Both check their boxes. CLOSE TRIP activates.

Scenarios:

Scenario A — Renter agrees to deduction:
Happy path with adjusted deposit amount.

Scenario B — Host enters PKR 0 deduction (decides not to charge for damage):
Allowed. Proceeds as no-damage path with full deposit return.

Scenario C — Host enters deduction larger than deposit:
Validation: "Deduction cannot exceed the security deposit of PKR 10,000." Error shown.

Scenario D — Renter refuses the deduction amount:
Taps "RAISE A DISPUTE". Navigates to Dispute Screen.

Flow 9.3 — CLOSE TRIP is Tapped

What the user sees:
Loading indicator. "Closing trip..."

What the app does:

Step 1: Upload post-trip photos to Firebase Storage
Path: inspections/{bookingId}/post_trip/{area}_{timestamp}.jpg

Step 2: Firestore transaction:

- Update bookings/{bookingId}:
    - status → "completed"
    - tripEndedAt → server timestamp
    - cashPayments.rentPaidToHost → true
    - cashPayments.rentPaidAmount → totalRent
    - cashPayments.rentPaidAt → server timestamp
    - cashPayments.depositRefundedToRenter → true
    - cashPayments.depositRefundedAmount → (deposit minus deduction)
    - cashPayments.depositRefundedAt → server timestamp
    - cashPayments.damageDeduction → deduction amount (0 if none)
    - updatedAt → server timestamp
- Write bookings/{bookingId}/inspections/post_trip — full post-trip data with photo URLs and comparison results
- Remove date range from cars/{carId}/bookedDateRanges — the car is now available for new bookings
- If a RozRides driver was used: update rozridesDrivers/{driverId} — isAvailable: true, currentBookingId: null (future feature, included for completeness)

- Write timeline entry: status "completed", note "Trip completed. Rent PKR [amount] collected. Deposit PKR [amount] refunded. Damage deduction: PKR [amount]."

Step 3: Post system message to conversation
" ✅ Trip completed on Jan 17 at 10:45 AM. All cash settled. Thanks for using RozRides!"
Mark conversation isActive: false (read-only mode)

Step 4: Schedule review prompts
Write two documents to scheduledNotifications:

text

```
{
  userId: renterId,
  type: "review_prompt",
  sendAt: now + 2 hours,
  bookingId: bookingId,
  sent: false
}
{
  userId: hostId,
  type: "review_prompt",
  sendAt: now + 2 hours,
  bookingId: bookingId,
  sent: false
}
```

A Cloud Function checks scheduledNotifications every 30 minutes and sends any where sendAt <= now AND sent == false, then marks sent: true.

Step 5: Navigate to completion screen
Show a success screen: "Trip Completed! Thanks for using RozRides." with "Leave a Review" and "Back to Home" buttons.

Error Scenarios:

Scenario A — Car date range removal fails:
Critical issue. Car remains blocked. Wrap everything in the transaction. If removal fails, entire transaction rolls back. User sees error and retries. Log to admin alerts.

Scenario B — Transaction succeeds but review schedule write fails:
Non-critical. Review prompts just won't be sent. Users can still manually leave reviews from the booking detail screen. Acceptable degraded state.

SECTION 10 — DISPUTES

Flow 10.1 — Renter Raises a Dispute

What the user sees:
DisputeScreen. Title: "Raise a Dispute"

Information card explaining the process.

"What happened from your perspective:" — large text area, minimum 50 characters required.

"Host is claiming a deduction of: PKR [amount]"
"In your view, the fair deduction is: PKR [input]"

"SUBMIT DISPUTE" button.

What the app does:

1. Creates damageClaims/{autoId}:

text

claimId: autoId

bookingId: bookingId

carId: carId

hostId: hostId

renterId: renterId

raisedBy: renterId

claimType: "damage_dispute"

description: renter's text

hostClaimedDeduction: amount host entered

renterAgreedDeduction: amount renter entered

status: "open"

createdAt: ServerTimestamp

2. Updates bookings/{bookingId}:
    - status → "disputed"
    - updatedAt → server timestamp
3. Writes timeline entry: status "disputed"
4. Writes to adminAlerts collection: "New dispute: Booking [id] — Host claims PKR [X], Renter claims PKR [Y]"
5. Sends notification to host: "[Renter name] has raised a dispute. RozRides admin will review and contact you."
6. Shows renter: "Dispute submitted. RozRides will review within 24 hours."

Flow 10.2 — Admin Reviews Dispute

Admin opens /dashboard/claims**:**

Sees the claim in a table. Clicks to open detail view showing:

- Full booking information
- Pre-trip photos for each area
- Post-trip photos for each area in comparison layout
- Renter's description
- Both claimed amounts
- The chat conversation log for this booking
- Admin notes text area
- Resolution buttons

Admin makes a decision:

Option A: "Resolve for Host" — damage is real, host's deduction is fair
Option B: "Resolve for Renter" — damage claim is unfair, full deposit returned
Option C: "Resolve Mutually" — admin enters a middle-ground amount

What happens on resolution:

Admin clicks resolution. A Firebase Cloud Function (triggered from admin action, using Admin SDK):

1. Updates damageClaims/{claimId}:
    - status → "resolved_for_host" or "resolved_for_renter" or "resolved_mutually"
    - adminNotes → admin's notes
    - resolvedAt → server timestamp
    - resolvedBy → adminId
2. Updates bookings/{bookingId}:
    - status → "completed" (was "disputed")
    - Updates final cashPayments with the resolved amounts
3. Updates reputation:
    - "Resolved for host": adds "damage_incident" note to renter's user document
    - "Resolved for renter": adds "disputed_deposit" note to host's user document. If host has 2+, writes an admin alert for account review.
4. Sends notification to both parties with the decision.

SECTION 11 — REVIEWS

Flow 11.1 — Review Prompt Arrives

2 hours after trip completion:
Renter notification: "How was your trip? Rate Toyota Corolla and Ahmed to help the community."
Host notification: "How was Ali Hassan as a renter? Leave a review."

Both notifications deep-link to SubmitReviewScreen with appropriate type.

Flow 11.2 — Renter Submits Review

What the renter sees:
Submit Review Screen showing the car photo and host photo. 5-star selector. Text field for comments. Submit button.

What the app does:

1. Checks if review already exists: queries reviews where bookingId == thisBookingId AND reviewerId == currentUserId. If found, shows "You've already reviewed this trip."
2. Writes review document: isPublic: false initially
3. Updates bookings/{bookingId}/reviewStatus/renterSubmitted: true
4. Checks if hostSubmitted is also true:
    - If yes: sets both reviews to isPublic: true and updates aggregate ratings
    - If no: does nothing, waits for host
5. Sends notification to host: "Ali Hassan left you a review! Leave your review to see theirs."

The 7-day window:
If after 7 days one party hasn't reviewed, the other's review is published anyway.
The scheduledNotifications system handles this with a document set for now + 7 days.

Flow 11.3 — Aggregate Rating Update

When both reviews are published OR 7-day window closes:

For car rating update:

1. Query all public renter_to_host reviews for this carId
2. Calculate average of all overallRating fields
3. Count total reviews
4. Build breakdown map: count how many are 1-star, 2-star, etc.
5. Write back to cars/{carId}: averageRating, totalReviews, ratingBreakdown

For user rating update (host's rating):

1. Query all public renter_to_host reviews where revieweeId == hostId
2. Calculate average
3. Write to users/{hostId}: hostRating, hostReviewCount

For renter's rating:

1. Query all public host_to_renter reviews where revieweeId == renterId
2. Calculate average
3. Write to users/{renterId}: renterRating, renterReviewCount